ĐẠI HỌC QUỐC GIA HÀ NỘI

TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN

KHOA TOÁN — CƠ — TIN HỌC

BÁO CÁO DỰ ÁN

HUẤN LUYỆN VÀ ĐÁNH GIÁ MÔ HÌNH YOLO11s CHO PHÁT HIỆN THỰC PHẨM VIỆT NAM TRÊN DỮ LIỆU VIETFOOD68

Lĩnh vực	Trí tuệ nhân tạo – Thị giác máy tính
Sinh viên thực hiện	Nguyễn Ngọc Kiều Trinh
Lớp	K70A2 – Khoa Toán – Cơ – Tin học
Đơn vị	Trường Đại học Khoa học Tự nhiên, Đại học Quốc gia Hà Nội
Giảng viên hướng dẫn	Chưa được cung cấp
Ngày cập nhật thực nghiệm	12/10/2026
Mã lần huấn luyện	vietfood68_5070-2

Hà Nội, tháng 10 năm 2026

Thông tin sinh viên và đơn vị được giữ theo bản báo cáo ban đầu. VietFood68 là tên dùng trong dự án; nguồn được dẫn ban đầu có tên công bố **VietFood67**, với 68 lớp gồm cả lớp người. Báo cáo sử dụng ID lớp của dữ liệu trên máy; chưa xác nhận đầy đủ bảng ánh xạ tên lớp của bản dữ liệu này.

TÓM TẮT

Đề tài xây dựng quy trình huấn luyện, suy luận và đánh giá phát hiện đối tượng bằng **YOLO11s**, sử dụng dữ liệu ảnh và nhãn YOLO tại `D:\archive\dataset`. Bộ dữ liệu thực nghiệm có 161.733 ảnh train, 6.602 ảnh validation và 3.284 ảnh test, với 68 ID lớp. Mô hình được khởi tạo từ trọng số tiền huấn luyện `yolo11s.pt` và hoàn tất 150 epoch trên NVIDIA GeForce RTX 5070 12 GB. Quy trình có điều chỉnh batch và bộ nạp dữ liệu để xử lý tình trạng thiếu RAM trên Windows.

Trọng số tốt nhất theo mAP50–95 trên validation nằm ở epoch 150. Đánh giá riêng toàn bộ test cho **Precision 95,34%, Recall 94,54%, mAP50 97,23% và mAP50–95 85,32%**. Với ngưỡng confidence 0,5936 được chọn trên validation rồi giữ cố định cho test, phép đếm ở IoU 0,50 thu được 5.316 TP, 222 FP và 354 FN, tương ứng precision gộp 95,99% và recall gộp 93,76%.

Kiểm tra nhãn không phát hiện dòng sai cấu trúc, nhãn thiếu hoặc ID ngoài phạm vi. Kiểm tra SHA-256 không phát hiện ảnh giống hệt byte giữa các tập. Tuy nhiên, tên file có dấu vết ảnh xuất từ Roboflow và khung hình video; chưa kiểm chứng việc tách theo ảnh nguồn/video. Các kết quả vì vậy phản ánh chất lượng trên bản chia dữ liệu hiện có, chưa chứng minh khả năng tổng quát ngoài môi trường này. Sản phẩm hiện có là mô hình và công cụ dòng lệnh; giao diện Streamlit là thiết kế đề xuất, chưa triển khai.

Từ khóa: YOLO11s; phát hiện đối tượng; thực phẩm Việt Nam; VietFood67; RTX 5070; mAP; đánh giá test.

Mục lục

TÓM TẮT **1**

DANH MỤC TỪ VIẾT TẮT **4**

1. ĐẶT VẤN ĐỀ VÀ MỤC TIÊU **4**
1.1. Bài toán . . . 4
1.2. Mục tiêu và phạm vi . . . 4

2. NGHIÊN CỨU LIÊN QUAN VÀ CƠ SỞ LÝ THUYẾT **5**
2.1. Các công trình tham khảo . . . 5
2.2. Kiến trúc sử dụng: YOLO11s . . . 5
2.3. Chỉ số đánh giá . . . 6
2.4. Hàm mất mát . . . 6

3. DỮ LIỆU VÀ KIỂM TRA CHẤT LƯỢNG **6**
3.1. Nguồn và phạm vi xác minh . . . 6
3.2. Tổ chức dữ liệu . . . 7
3.3. Thống kê kiểm kê . . . 7
3.4. Kiểm tra chất lượng và trùng lặp . . . 7
3.5. Phân bố lớp . . . 8

4. PHƯƠNG PHÁP VÀ TỔ CHỨC HỆ THỐNG **8**
4.1. Luồng thực nghiệm . . . 8
4.2. Tiền xử lý và augmentation . . . 9
4.3. Các thành phần mã nguồn . . . 9

5. MÔI TRƯỜNG VÀ CẤU HÌNH THỰC NGHIỆM **9**
5.1. Phần cứng và phần mềm . . . 9
5.2. Cấu hình huấn luyện . . . 10
5.3. Tối ưu bộ nhớ và lưu trữ . . . 11
5.4. Thời gian và các lần tiếp tục train . . . 11

6. QUY TRÌNH ĐÁNH GIÁ **12**
6.1. Chọn mô hình và ngưỡng . . . 12
6.2. Cấu hình evaluator . . . 12
6.3. Quy tắc đọc confusion matrix . . . 13

7. KẾT QUẢ VÀ THẢO LUẬN **13**
7.1. Diễn biến huấn luyện . . . 13
7.2. Kết quả tổng thể . . . 14
7.3. Ảnh hưởng ngưỡng confidence . . . 14
7.4. Kết quả từng lớp . . . 15

7.5. Tốc độ suy luận 16

8. PHÂN TÍCH LỖI VÀ KIỂM THỬ **17**
8.1. Lỗi ở điểm vận hành đã khóa 17
8.2. Minh họa và nhận xét cụ thể 17
8.3. Ma trận nhầm lẫn tại ngưỡng vận hành 22
8.4. Kiểm thử và bằng chứng 22

9. THIẾT KẾ ỨNG DỤNG VÀ HƯỚNG PHÁT TRIỂN **23**
9.1. Thiết kế giao diện đề xuất 23
9.2. Thứ tự phát triển tiếp theo 24

10. HẠN CHẾ, KHẢ NĂNG TÁI LẬP VÀ KẾT LUẬN **24**
10.1. Hạn chế của kết quả 24
10.2. Tái lập và quản lý hiện vật 25
10.3. Kết luận 25

11. TÀI LIỆU THAM KHẢO **25**

PHỤ LỤC A. KẾT QUẢ TEST ĐỦ 68 LỚP **26**

PHỤ LỤC B. LỆNH CHẠY VÀ TÁI ĐÁNH GIÁ **28**

PHỤ LỤC C. DANH MỤC HIỆN VẬT **29**

PHỤ LỤC D. THÔNG TIN CHƯA THỂ TỰ ĐIỀN **29**

DANH MỤC TỪ VIẾT TẮT

Ký hiệu	Ý nghĩa
YOLO	You Only Look Once
GT	Ground truth: đối tượng đã được gán nhãn
TP / FP / FN	Phát hiện đúng / phát hiện sai / đối tượng bị bỏ sót theo quy tắc đánh giá
IoU	Intersection over Union: độ giao nhau trên hợp của hai hộp
AP / mAP	Average Precision / trung bình AP giữa các lớp
NMS	Non-Maximum Suppression: loại bỏ dự đoán chồng lặp
AMP	Automatic Mixed Precision: tính toán với nhiều mức chính xác
DFL	Distribution Focal Loss
VRAM	Bộ nhớ riêng của GPU

---

1. ĐẶT VẤN ĐỀ VÀ MỤC TIÊU

1.1. Bài toán

Một ảnh thực phẩm có thể chứa nhiều món, nguyên liệu và đối tượng phụ trợ. Phân loại toàn ảnh chỉ cung cấp nhãn ảnh, trong khi phát hiện đối tượng cần trả về đồng thời **vị trí hộp, ID lớp và độ tin cậy** cho từng đối tượng. Đề tài lựa chọn bài toán thứ hai để hỗ trợ quan sát món ăn và thành phần trong ảnh.

Khó khăn thực tế gồm vật thể nhỏ, che khuất, thành phần nằm sát nhau, khác biệt góc nhìn và quy tắc gán nhãn không đồng nhất. Mô hình còn cần chạy ổn định trên máy cá nhân, nơi tài nguyên GPU, RAM và dung lượng đĩa đều có giới hạn. Vì vậy, chất lượng đánh giá và ổn định vận hành được xem là hai yêu cầu đồng thời.

1.2. Mục tiêu và phạm vi

Mục tiêu	Tiêu chí kiểm chứng	Kết quả thực hiện
Chuẩn bị dữ liệu YOLO 68 lớp	Kiểm kê ảnh, nhãn, ID và định dạng	Đã kiểm tra toàn bộ nhãn
Huấn luyện trên RTX 5070	Hoàn tất số epoch và lưu checkpoint	Hoàn tất 150 epoch
Đánh giá mô hình	Tách số liệu validation và test	Đã đánh giá riêng cả hai tập
Phân tích chất lượng	Kết quả từng lớp, FP/FN, hình minh họa	Có CSV, JSON và biểu đồ kèm báo cáo
Suy luận trên ảnh	Chạy được mô hình đã huấn luyện	Đã chạy benchmark trên 100 ảnh test
Giao diện người dùng	Tải ảnh và hiển thị tên món đúng	Đã thiết kế; chưa có ứng dụng trong repo

Đóng góp thực tế của dự án là quy trình huấn luyện và đánh giá có thể kiểm tra lại, cùng cách giới hạn bộ nhớ bộ nạp dữ liệu trên Windows. Đề tài sử dụng kiến trúc có sẵn, không đề xuất kiến trúc YOLO mới. Nhận diện khẩu phần, tính calo, khảo sát người dùng và triển khai web chưa thuộc kết quả đã hoàn thành.

2. NGHIÊN CỨU LIÊN QUAN VÀ CƠ SỞ LÝ THUYẾT

2.1. Các công trình tham khảo

YOLO gốc của Redmon và cộng sự biểu diễn phát hiện đối tượng bằng dự đoán hộp và xác suất lớp trong một mạng, tạo nền tảng cho họ mô hình phát hiện một giai đoạn. Kiến trúc YOLO11 trong đề tài là thế hệ triển khai của Ultralytics, không đồng nhất với kiến trúc YOLO gốc. 1

Công trình	Bài toán và dữ liệu	Giá trị tham khảo cho đề tài
Redmon và cộng sự, YOLO (2016)	Phát hiện đối tượng	Cách tiếp cận dự đoán hộp và lớp trong một mạng [1]
Nguyen và cộng sự, VinaFood21 (2021)	Nhận dạng thực phẩm; 13.950 ảnh, 21 loại món	Bối cảnh nghiên cứu ảnh thực phẩm Việt Nam; khác bộ dữ liệu và bài toán [2]
Nguyen và cộng sự, VietFood67 / FoodDetector (2025)	33.003 ảnh, 68 lớp có cả khuôn mặt người; YOLOv10	Nguồn liên quan trực tiếp đến tên bộ dữ liệu; mô hình tham khảo công bố mAP50 0,92 [3]
Ultralytics YOLO11 (2024)	Họ mô hình phát hiện và các tác vụ thị giác khác	Kiến trúc và thư viện thực nghiệm được sử dụng [4]

VinaFood21 nghiên cứu nhận dạng món ăn với CNN. Kết quả accuracy của phân loại không so sánh trực tiếp với mAP của phát hiện đối tượng. 2

Bài báo VietFood67 cung cấp bối cảnh phù hợp hơn vì có nhãn hộp và lớp người. Tuy nhiên, báo cáo này đo trên 171.619 file ảnh đã có trên máy, lớn hơn số ảnh gốc công bố. Chưa có quy trình tạo bản dữ liệu này hoặc cùng giao thức chia tập để đối chiếu. Vì vậy, mAP50 test 97,23% của dự án **không đủ cơ sở kết luận vượt YOLOv10 92% trong công trình tham khảo**. 3

2.2. Kiến trúc sử dụng: YOLO11s

YOLO11 có các biến thể n, s, m, l và x; đề tài dùng biến thể **s**. Trọng số khởi tạo `yolo11s.pt` hỗ trợ chuyển giao từ dữ liệu tiền huấn luyện sang bộ lớp mới. 4

Kiểm tra trực tiếp cấu hình trong checkpoint xác nhận `yaml_file=yolo11s.yaml`, `scale=s`, `nc=68`, ảnh RGB 3 kênh. Backbone chứa các khối Conv, C3k2, SPPF và C2PSA; phần head kết hợp đặc trưng qua upsample/concat và đưa ba mức đặc trưng vào Detect. Cách tổ chức backbone–neck–head và các khối này phù hợp mô tả kiến trúc chính thức. 5

Mô hình trong checkpoint có **9.454.108 tham số**; sau hợp nhất Conv/BatchNorm khi đánh giá còn **9.439.116 tham số**, được log là 21,6 GFLOPs. Trọng số `best.pt` có kích thước **19.226.330 byte**. Đây là số đo của mô hình 68 lớp đang sử dụng, không lấy kích thước mô hình 80 lớp trên COCO

thay cho thực nghiệm.

2.3. Chỉ số đánh giá

Với một ngưỡng IoU cố định, dự đoán đúng cần trùng lớp với GT và đủ mức chồng lấp, theo quy tắc ghép cặp một–một. Dự đoán không ghép được tính FP; GT không ghép được tính FN. Các công thức đếm:

```
IoU       = diện tích giao / diện tích hợp
Precision = TP / (TP + FP)
Recall    = TP / (TP + FN)
F1        = 2 × Precision × Recall / (Precision + Recall)
```

Precision cao cho biết phần lớn dự đoán được giữ là đúng; recall cao cho biết ít GT bị bỏ sót. Confidence là điểm mô hình xuất ra và ngưỡng lọc, không phải accuracy của toàn hệ thống. 6

AP tổng hợp đường precision–recall theo confidence cho từng lớp. **mAP50** lấy trung bình AP của các lớp ở IoU 0,50. **mAP50–95** tiếp tục lấy trung bình trên 10 mức IoU từ 0,50 đến 0,95, bước 0,05, nên nhạy hơn với độ chính xác vị trí hộp. Quy ước nhiều mức IoU được sử dụng trong đánh giá COCO; báo cáo này dùng cách tính AP của Ultralytics trên nhãn YOLO, không coi đó là kết quả trên tập COCO. 7

Macro là trung bình đều giữa các lớp có GT; **micro** cộng TP/FP/FN trên tất cả lớp rồi tính tỷ lệ. Các lớp nhiều mẫu chi phối micro mạnh hơn. Báo cáo ghi rõ cách tổng hợp và ngưỡng confidence khi so sánh kết quả.

2.4. Hàm mất mát

Theo log thực nghiệm, mô hình tối ưu ba thành phần: `box_loss` cho định vị hộp, `cls_loss` cho dự đoán lớp và `dfl_loss` cho biểu diễn phân phối tọa độ hộp. Cấu hình hệ số tương ứng là 7,5; 0,5 và 1,5. Các giá trị loss không phải phần trăm lỗi; không so trực tiếp trị số box loss với cls loss để kết luận thành phần nào tốt hơn. Xu hướng giảm qua epoch có ý nghĩa hơn giá trị đơn lẻ.

Loss train và validation được tính trong điều kiện khác nhau, đặc biệt train có augmentation. Khoảng cách giữa hai đường vì vậy cần đọc cùng mAP và quy trình dữ liệu, không tự động đồng nghĩa với overfitting.

3. DỮ LIỆU VÀ KIỂM TRA CHẤT LƯỢNG

3.1. Nguồn và phạm vi xác minh

Bản báo cáo ban đầu dẫn trang Kaggle `thomasnguyen6868/vietfood68`, hiện được công bố dưới tên **VietFood67**. Repo FoodDetector của tác giả mô tả 68 lớp, trong đó có lớp người, và cung cấp danh sách tên tham chiếu. 8, 9

Trong bản dữ liệu trên máy chỉ tìm thấy thư mục ảnh và nhãn; `data.yaml` của dự án hiện đặt tên `class_0` đến `class_67`. Chưa có manifest phiên bản tải xuống, cấu hình xuất Roboflow hay bảng tên lớp gốc kèm theo. Do đó cần phân biệt **nguồn được dẫn** với **nguồn gốc đã kiểm chứng của từng file hiện có**.

Đối chiếu danh sách `class_names.py` từ repo tác giả cho thấy các ID tham chiếu 8, 27, 47, 50, 52 lần lượt là bò kho, người, phô mai, thịt bò, thịt heo. Những ID này phù hợp với các mẫu minh họa đã quan sát. Đây là bằng chứng hỗ trợ thứ tự lớp, chưa thay thế xác minh toàn bộ 68 ID của bản dữ liệu cục bộ. Không gán lại tên trong checkpoint chỉ dựa vào tên bộ dữ liệu. 10

3.2. Tổ chức dữ liệu

```
D:\archive\dataset\
├── images\
│   ├── train\
│   ├── valid\
│   └── test\
└── labels\
    ├── train\
    ├── valid\
    └── test\
```

Mỗi ảnh có một file `.txt` cùng stem. Một dòng nhãn có dạng:

```
class_id x_center y_center width height
```

Các tọa độ được chuẩn hóa theo chiều rộng và chiều cao ảnh. File nhãn rỗng biểu diễn ảnh không có đối tượng được gán nhãn, không mặc định là file hỏng. Nhãn hợp lệ về số học chưa chứng minh đối tượng đã được gán đầy đủ hoặc chính xác về ngữ nghĩa.

3.3. Thống kê kiểm kê

Bảng 1. Số lượng ảnh, nhãn và đối tượng trong bản dữ liệu thực nghiệm.

Tập	Ảnh	File nhãn	Hộp GT	Nhãn rỗng	ID có mặt
Train	161.733	161.733	870.943	630	68
Validation	6.602	6.602	11.582	168	68
Test	3.284	3.284	5.670	75	68

Tổng số file ảnh là **171.619**, tổng số hộp gán nhãn là **888.195**. Số ảnh train chiếm 94,24%; validation 3,85%; test 1,91%. Đây là tỷ lệ file hiện có, không phải tỷ lệ ảnh nguồn độc lập. Kích thước tổng file ảnh khoảng **24,55 GiB**; kích thước sau giải mã trong RAM có thể lớn hơn nhiều.

Train trung bình 5,39 hộp/ảnh, trong khi validation 1,75 và test 1,73 hộp/ảnh. Khác biệt này cần được đối chiếu với quy trình augmentation/xuất dữ liệu trước khi xem ba tập có cùng phân phối. Báo cáo không suy ra số ảnh gốc từ số file train.

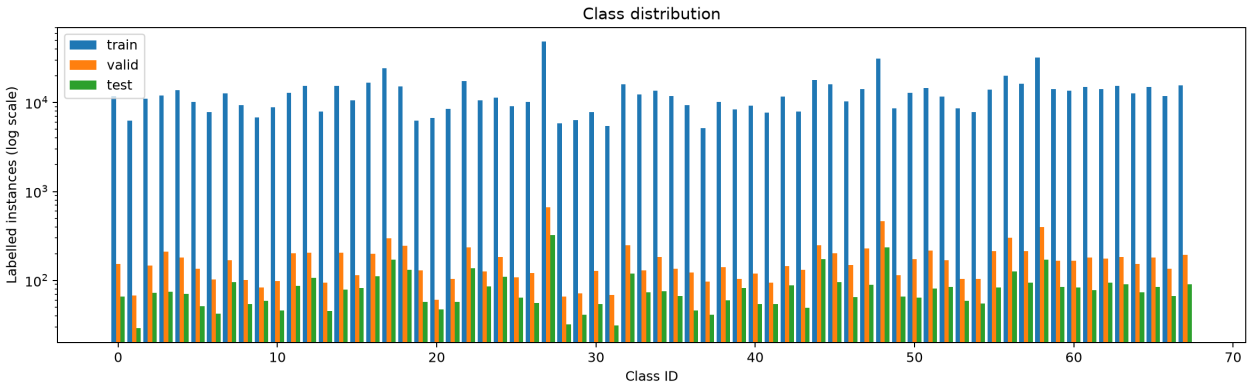
3.4. Kiểm tra chất lượng và trùng lặp

Script `audit_dataset.py` kiểm tra toàn bộ file nhãn: mỗi dòng phải có 5 giá trị hữu hạn; ID nguyên từ 0 đến 67; tọa độ nằm trong [0,1]; chiều rộng và cao dương. Kết quả: **0 dòng sai, 0 nhãn thiếu, 0 nhãn không có ảnh tương ứng**; cả ba tập đều có đủ 68 ID. Phép kiểm tra này chưa kiểm chứng tính nhất quán ngữ nghĩa, các hộp nằm hoàn toàn trong ảnh hay mọi trường hợp hộp lặp. Evaluator đã đọc đủ 6.602 ảnh validation và 3.284 ảnh test, không báo ảnh lỗi trong hai tập này.

Kiểm tra trùng byte dùng SHA-256 cho tất cả ảnh validation/test và các ảnh train có kích thước file trùng ứng viên. Tổng cộng **17.990 file** được băm. Những file khác kích thước không thể giống hệt byte nên được loại an toàn trước khi băm. Kết quả có **0 nhóm ảnh giống hệt byte giữa các tập**.

Sau khi bỏ hậu tố `.rf.`, có 336 tên gốc trùng giữa train–validation và 298 giữa train–test; tương ứng 4.747 file validation và 2.330 file test mang tên gốc có xuất hiện ở train. Nhiều tên chỉ là `10 0_jpg`, `101_jpg` nên trùng tên **không chứng minh trùng ảnh nguồn**. Ngược lại, SHA-256 khác nhau cũng không loại được ảnh gần trùng, ảnh biến đổi hoặc khung hình cùng video. Cần manifest nguồn và kiểm tra tương đồng ảnh trước khi xác nhận việc chia tập độc lập.

3.5. Phân bố lớp

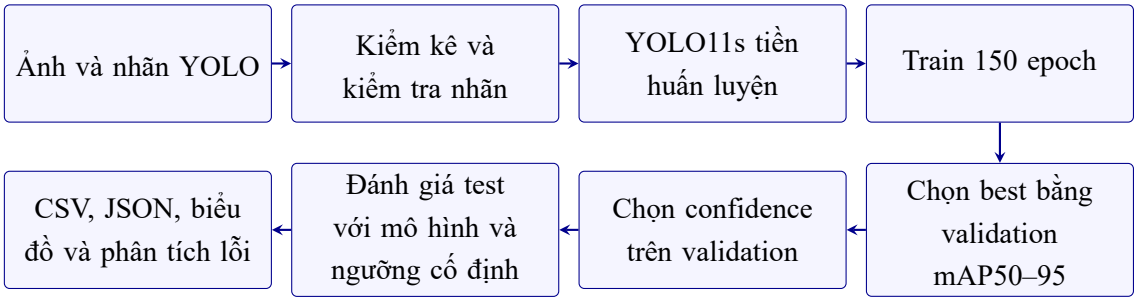


Hình 1. Mỗi cột biểu diễn số hộp, không phải số ảnh; một ảnh có thể đóng góp cho nhiều lớp.

Số hộp train theo lớp nằm trong khoảng 5.193–48.477, chênh lệch khoảng 9,34 lần. Test chỉ có 29–323 hộp mỗi lớp. Bên cạnh mAP trung bình, cần theo dõi lớp ít mẫu và lớp có AP thấp; biến động một vài dự đoán có thể ảnh hưởng lớn đến tỷ lệ của lớp ít mẫu. Số liệu từng lớp được lưu tại dataset_class_counts.csv.

4. PHƯƠNG PHÁP VÀ TỔ CHỨC HỆ THỐNG

4.1. Luồng thực nghiệm



Test chỉ được dùng đánh giá sau khi training hoàn tất. Trong lượt đánh giá báo cáo, không dùng test để chọn epoch hoặc điều chỉnh trọng số. Ngưỡng vận hành được chọn từ đường F1 của validation; mAP vẫn được tính từ dự đoán ở confidence sàn thấp để bảo toàn đường precision–recall.

4.2. Tiền xử lý và augmentation

Train dùng ảnh RGB, `imgsz=640` và `rect=False`. Thư viện xử lý resize/letterbox và hộp tương ứng. Cấu hình augmentation thực tế gồm HSV (`h=0,015`, `s=0,7`, `v=0,4`), dịch chuyển `translate=0,1`, thay đổi tỷ lệ `scale=0,5`, lật ngang `fliplr=0,5` và mosaic với xác suất 1,0 trước giai đoạn đóng mosaic. Xoay, shear, perspective, lật dọc, mixup, cutmix và copy-paste đặt bằng 0.

`close_mosaic=10` đóng mosaic trong 10 epoch cuối, từ epoch 141. Hình train tại giai đoạn này được lưu trong thư mục run. Những trường như `auto_augment` và `erasing` có thể xuất hiện trong cấu hình chung của thư viện; báo cáo không coi chúng là augmentation detection đã áp dụng chỉ vì có trong YAML.

Validation/test không dùng tăng cường dữ liệu lúc đánh giá (`augment=False`). Lượt đánh giá dùng `rect=True` để ghép theo tỷ lệ ảnh và căn theo stride. Vì vậy `imgsz=640` là kích thước cấu hình, không bảo đảm mọi tensor đều đúng 640×640; hình vuông có thể được padding lên 672×672 trong evaluator phiên bản đang dùng. Suy luận benchmark sử dụng `rect=False`, tensor cố định 640×640. [11]

4.3. Các thành phần mã nguồn

File	Vai trò
data.yaml	Đường dẫn ba tập và danh sách ID lớp
check_gpu.py	Kiểm tra CUDA và phép toán có backward
train.py	Tham số train, AMP, AutoBatch, resume và đường dẫn output
memory_trainer.py	Giới hạn worker/prefetch; validation dùng 0 worker
evaluate.py	Đánh giá validation/test, chọn ngưỡng và lưu số liệu
audit_dataset.py	Kiểm kê nhãn và trùng ảnh byte
analyze_results.py	Biểu đồ và phân tích lỗi từng ảnh
benchmark_inference.py	Đo suy luận từng ảnh trên mẫu test cố định
render_report.py	Điền số đo và bảng vào mẫu báo cáo

5. MÔI TRƯỜNG VÀ CẤU HÌNH THỰC NGHIỆM

5.1. Phần cứng và phần mềm

Bảng 2. Môi trường đo trực tiếp trên máy chạy.

Thành phần	Giá trị
Hệ điều hành	Windows 11 Pro, build 26200; chạy Windows native
GPU	NVIDIA GeForce RTX 5070 desktop, 12 GB
CPU	AMD Ryzen 5 7600X3D; 6 lõi vật lý, 12 luồng logic
RAM	32 GB lắp đặt; hệ điều hành báo khoảng 31,11 GiB khả dụng tổng

Thành phần	Giá trị
Python	3.12.13
Ultralytics	8.4.175
PyTorch / torchvision	2.8.0+cu128 / 0.23.0+cu128
CUDA runtime của PyTorch	12.8
Driver NVIDIA	617.14
OpenCV	5.0.0.93
Nơi lưu dataset / run	Ổ D; môi trường `.venv` trong repo là junction sang D

CUDA runtime của PyTorch khác phiên bản CUDA tối đa driver hỗ trợ. Không cần diễn giải số CUDA do `nvidia-smi` hiển thị là phiên bản runtime của training. Phiên bản thư viện trước và sau đánh giá được chụp tại environment-freeze.txt và environment-freeze-final.txt. Lượt `save_json=True` tự bổ sung `faster-coco-eval==1.8.0`; không thay đổi phiên bản PyTorch, Ultralytics hoặc các thư viện training đang có. Các bảng mAP chính vẫn lấy từ evaluator native, không trộn thêm chỉ số COCO theo kích thước vật thể.

5.2. Cấu hình huấn luyện

Bảng 3. Cấu hình cuối được đối chiếu với `args.yaml` và mã nguồn.

Tham số	Cấu hình thực nghiệm
Model / khởi tạo	YOLO11s / `yolo11s.pt` tiền huấn luyện
Số lớp	68
Epoch	150; hoàn tất đủ 150
Kích thước train	640×640, `rect=False`
Batch	28 ở giai đoạn đầu; 36 từ epoch 73
Train workers / prefetch cuối	4 worker / 2 batch mỗi worker
Validation workers trong train	0, do trainer tùy chỉnh
Image cache	Tắt (`cache=False`)
Device / AMP	CUDA:0 / bật
Optimizer thực tế	MuSGD, được chọn từ chế độ `auto`
`lr0`, `lrf`, momentum	0,01; 0,01; 0,937 trong cấu hình lưu
Weight decay / warmup	0,0005 / 3 epoch
Lịch learning rate	Cosine (`cos_lr=True`)
`nbs`	64; batch thực có thể kết hợp gradient accumulation
Seed / deterministic	42 / False
Early stopping	`patience=30`
Đóng mosaic	10 epoch cuối
Lưu định kỳ	`save_period=10`; ngoài `best.pt` và `last.pt`

MuSGD được ghi trong cấu hình cuối, không thay bằng AdamW khi mô tả. Chế độ `auto` của thư viện có thể tự chọn optimizer và learning rate; các giá trị cấu hình không thay thế việc kiểm tra log learning rate thực tế. Cột `lr/pg*` đã được lưu trong `results.csv`. 12

`nbs=64` không có nghĩa batch vật lý luôn 64. Với quy tắc tích lũy của phiên bản đang dùng, batch 28 cho khoảng 56 ảnh mỗi bước cập nhật sau warmup, batch 36 cho khoảng 72. Thay batch có thể thay đổi đường tối ưu. Vì các thay đổi xảy ra trong cùng run, không thể quy toàn bộ mức tăng precision cho batch lớn hơn.

5.3. Tối ưu bộ nhớ và lưu trữ

Trong quá trình chạy, máy từng có RAM dùng khoảng 30,4 GB và committed memory 61,5/62,5 GB. Lỗi OpenCV báo không cấp phát được khoảng 6,2 MB xảy ra ở DataLoader worker; đây là tình trạng bộ nhớ hệ thống, không phải chỉ nhìn VRAM để chẩn đoán.

DataLoader worker chuẩn bị ảnh ở phía CPU; nhiều worker và hàng đợi prefetch làm nhiều batch tồn tại đồng thời. Trên Windows, các tiến trình có thể giữ bản sao cấu trúc dataset; train và validation có thể cùng giữ pool. Tài liệu PyTorch mô tả rõ mối liên hệ worker, prefetch và bộ nhớ tiến trình. 13

Giải pháp áp dụng trong repo: tắt cache ảnh RAM, giảm/kiểm soát prefetch, giữ validation 0 worker và tăng train worker từng bước. Hồ sơ cuối có 4×2=8 batch prefetch danh nghĩa, ngoài batch đang xử lý và các buffer khác. AMP được bật để giảm một phần chi phí tính toán/bộ nhớ GPU; AMP không giới hạn tổng RAM của worker. 14

Dataset, run và môi trường Python được chuyển sang D. Tuy nhiên, chuyển output không đồng nghĩa mọi file hệ thống chuyển theo: pagefile và một số thư mục tạm vẫn có thể dùng C. Báo cáo không xác nhận đã di chuyển pagefile. Cần theo dõi RAM committed và dung lượng C cùng GPU memory.

5.4. Thời gian và các lần tiếp tục train

CSV có đủ 150 epoch nhưng cột `time` bị đặt lại ở các lần resume. Không dùng giá trị `time` cuối như tổng thời gian chạy từ đầu và không cộng mù mọi chênh lệch âm/dương. Trong đoạn ổn định **epoch 74–150**, chênh lệch thời gian giữa hai dòng liên tiếp có trung vị **13,36 phút/epoch**, khoảng **13,29–13,72 phút**. Đây là thời gian vòng epoch ghi trong CSV, có phần validation và thao tác cuối epoch.

Nếu giữ tốc độ trung vị này từ đầu, 150 epoch tương đương khoảng **33,40 giờ**. Con số này là ước lượng theo tốc độ của đoạn ổn định, không phải tổng thời gian thực tế đã đo cho run có gián đoạn. Chưa có nhật ký đầy đủ để tổng hợp thời gian nghỉ, lần chạy dở và mọi lần restart.

Early stopping theo `patience=30` sẽ dừng khi tiêu chí theo dõi không cải thiện đủ lâu. Run này kết thúc do đạt 150 epoch, không dừng sớm. Sau kết thúc bình thường, checkpoint đã được strip optimizer; không dùng `resume=True` để tiếp tục trạng thái tối ưu đã bị loại khỏi `best.pt`/`last.pt` hiện tại.

6. QUY TRÌNH ĐÁNH GIÁ

6.1. Chọn mô hình và ngưỡng

1. Chọn `best.pt` theo fitness validation của phiên bản đang dùng. Kiểm tra mã `Metric.fitness` cho trọng số `[0,0,0,1]` với thứ tự P, R, mAP50, mAP50–95, nên tiêu chí chọn là mAP50–95.
2. Đọc toàn bộ CSV: epoch 150 có mAP50–95 cao nhất. Không chọn checkpoint bằng kết quả test.
3. Đánh giá lại best trên toàn bộ validation; chọn confidence **0,5936** tại cực đại của đường F1 trung bình đã làm trơn theo evaluator.
4. Đánh giá test với cùng trọng số và cấu hình. Giữ confidence đã chọn trên validation khi báo cáo điểm vận hành.

Evaluator native vẫn xuất P/R ở confidence tối ưu F1 riêng của mỗi tập. Ngưỡng native của test là **0,5676**, chỉ dùng diễn giải bảng mặc định. Báo cáo đồng thời cung cấp điểm vận hành đã khóa theo validation để tránh trình bày việc tối ưu ngưỡng trên test như kết quả triển khai.

6.2. Cấu hình evaluator

Tham số	Giá trị
Dữ liệu	Toàn bộ validation 6.602 ảnh; toàn bộ test 3.284 ảnh
`imgsz`, `rect`	640, True
Batch / workers	16 / 0
Kiểu tính toán	FP16 trên CUDA:0
Confidence sàn khi tính AP	0,001
NMS IoU	0,70
IoU ghép cặp cho AP	0,50:0,05:0,95
`max_det`	300
Test-time augmentation	Tắt
Output	Đường P/R/F1, confusion matrix, predictions JSON, CSV từng lớp

NMS IoU 0,70 và IoU ghép cặp 0,50 có hai vai trò khác nhau. Không dùng 0,70 để diễn giải điều kiện TP trong bảng TP/FP/FN. Tham số `half=True` được thư viện chuyển sang FP16; phiên bản hiện tại cảnh báo tên tham số này sắp thay bằng `quantize=16`, nhưng lượt đánh giá đã hoàn thành bình thường.

Script giữ lại thống kê ghép cặp trước khi Ultralytics xóa chúng. Một lượt phân tích độc lập sau đó đọc predictions JSON, lọc theo confidence validation rồi ghép lại với nhãn gốc trên tọa độ ảnh. Các số TP/FP/FN trùng với phép đếm từ evaluator, giúp kiểm tra điểm vận hành. JSON làm tròn bbox đến 3 chữ số thập phân và score đến 5 chữ số; các trường hợp sát ngưỡng cần lưu ý sai số làm tròn.

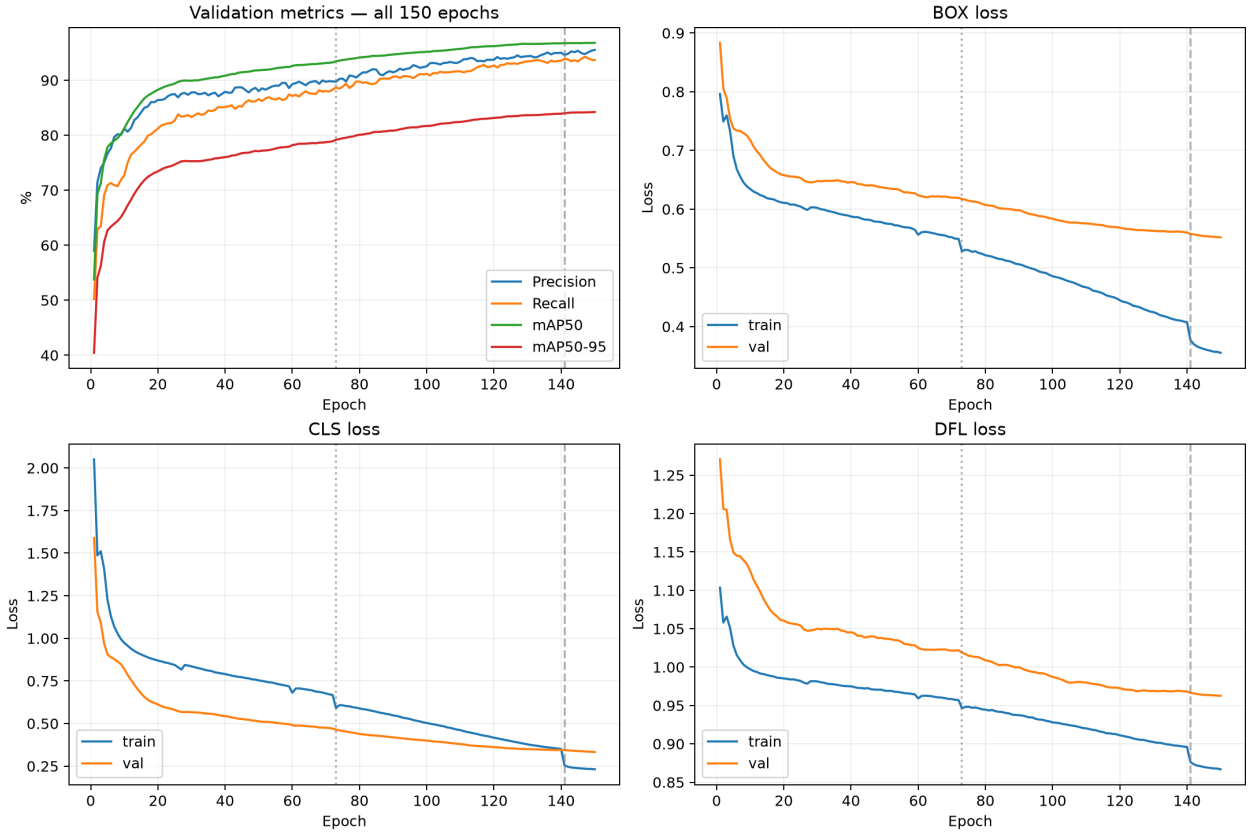
6.3. Quy tắc đọc confusion matrix

Ma trận mặc định của lượt val/test dùng confidence sàn **0,001** và IoU hình học **0,45**, nên không đại diện trực tiếp cho ngưỡng triển khai. Báo cáo tạo thêm ma trận tại **confidence 0,5936, IoU 0,50**. Hàng là lớp dự đoán, cột là GT; hàng/cột cuối biểu diễn background/bỏ sót.

Ghép cặp hình học của confusion matrix không bắt buộc cùng lớp trước khi ghép; cách ghép của AP có điều kiện cùng lớp. Do đó không bắt buộc tổng đường chéo của matrix bằng TP từ AP trong mọi trường hợp có hộp chồng nhau. Không lấy matrix mặc định để suy lại P/R ở confidence khác.

7. KẾT QUẢ VÀ THẢO LUẬN

7.1. Diễn biến huấn luyện



Hình 2. Vạch chấm tại epoch 73 đánh dấu batch 36; vạch đứt tại epoch 141 đánh dấu đóng mosaic. Các mốc biểu diễn thay đổi cấu hình, không chứng minh tác động nhân quả của chúng.

Bảng 4. Một số mốc từ log huấn luyện; chỉ số đều tính trên validation.

Epoch	Precision	Recall	mAP50	mAP50–95
1	59,00	50,20	53,77	40,43
10	81,19	72,73	81,25	66,10
25	87,49	82,69	89,39	74,67
50	88,03	86,58	91,81	77,09
72	89,79	88,30	93,24	78,91

Epoch	Precision	Recall	mAP50	mAP50–95
73	89,74	88,69	93,46	79,18
100	92,54	91,17	95,19	81,69
125	94,07	93,00	96,45	83,43
140	95,00	93,56	96,73	83,91
141	94,62	93,88	96,75	84,00
150	95,53	93,67	96,81	84,23

Từ epoch 1 đến 150, precision tăng **36,53 điểm phần trăm**, recall tăng **43,47 điểm** và mAP50–95 tăng **43,80 điểm**. Precision có dao động cục bộ; giảm ở một epoch không đủ kết luận mô hình suy giảm dài hạn. Ở giai đoạn cuối, chỉ số vẫn cải thiện nhưng mức tăng nhỏ dần.

Epoch 150 có train box/cls/DFL loss lần lượt **0,35527 / 0,23118 / 0,86684**; validation tương ứng **0,55206 / 0,33274 / 0,96275**. Val box loss và cls loss tiếp tục giảm cùng mAP tăng, nên chưa thấy dấu hiệu suy giảm validation kéo dài trong run này. Tuy nhiên, chưa kiểm chứng nguồn chia tập, chưa có nhiều seed hoặc tập ngoài miền để loại trừ overfitting hay ước lượng bất định.

7.2. Kết quả tổng thể

Bảng 5. So sánh log cuối, đánh giá lại validation và test. Đơn vị chỉ số: %.

Nguồn	Ảnh	Precision	Recall	mAP50	mAP50–95
Log epoch 150 (validation)	6.602	95,53	93,67	96,81	84,23
Validation đánh giá lại	6.602	95,58	93,66	96,80	84,22
Test sau training	3.284	95,34	94,54	97,23	85,32

Số validation đánh giá lại có chênh lệch nhỏ với log cuối do điều kiện evaluator như batch và tính toán FP16; mức chênh lệch không làm thay đổi kết luận về run. Test có mAP50–95 cao hơn validation khoảng **1,09 điểm phần trăm**. Không diễn giải điều đó là test tốt hơn mọi tình huống thực tế; độ khó, phân bố đối tượng và mẫu của từng tập có thể khác nhau.

Kết quả test 95,34% precision cho thấy mục tiêu trên 95% đã đạt theo **cách tổng hợp native của evaluator**. Đây không phải tỷ lệ ảnh nhận diện đúng hoàn toàn, không phải precision của mọi lớp, và không phải bảo đảm cho ảnh mới ngoài bộ dữ liệu.

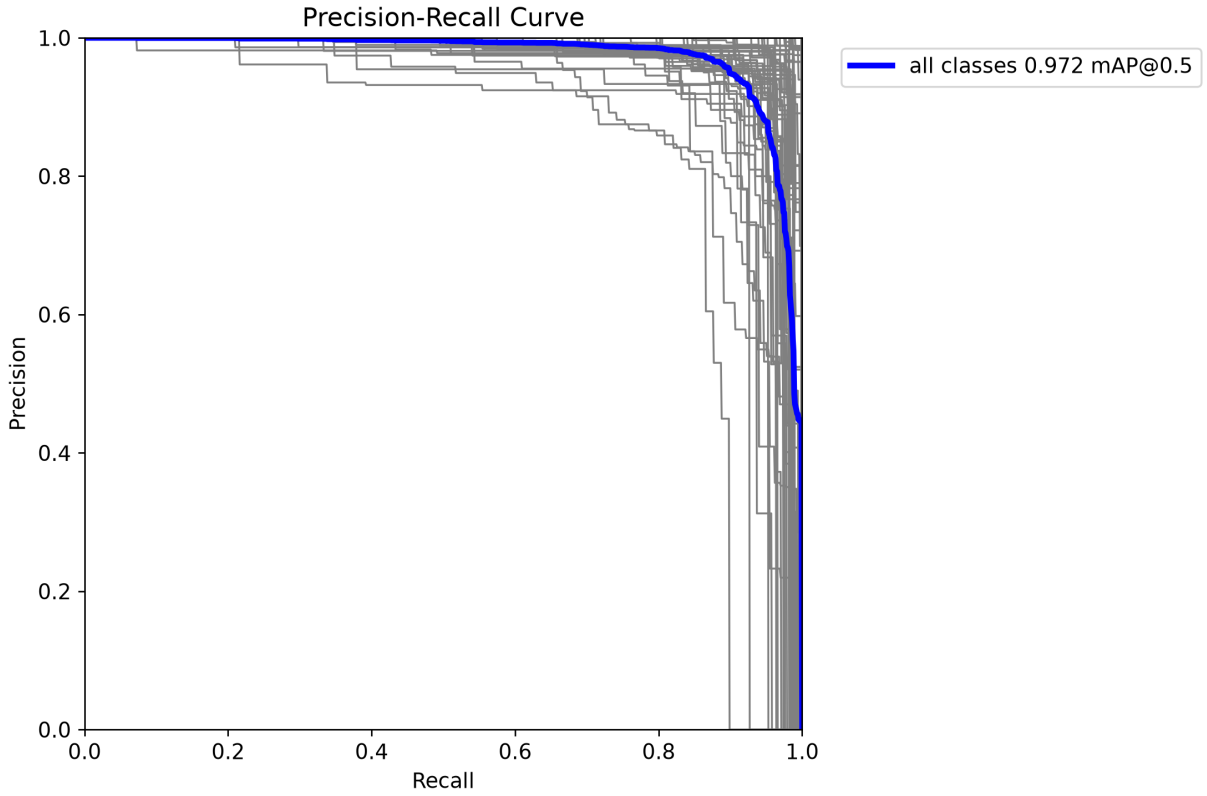
7.3. Ảnh hưởng ngưỡng confidence

Bảng 6. TP/FP/FN test ở IoU 0,50; P/R là micro.

Confidence	TP	FP	FN	Precision micro	Recall micro
0,2500	5455	508	215	91,48	96,21
0,5936	5316	222	354	95,99	93,76

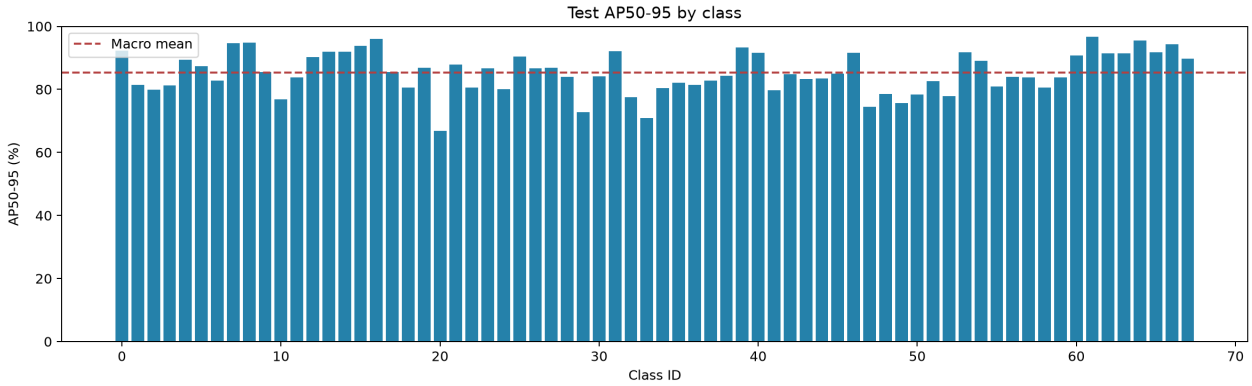
Ngưỡng khóa theo validation giảm FP từ 508 xuống 222 so với 0,25, đồng thời tăng FN từ 215 lên 354. Mức đánh đổi này phù hợp khi ưu tiên ít phát hiện sai; hệ thống cần quan sát đầy đủ thành phần có thể chọn ngưỡng thấp hơn và chấp nhận thêm FP. Việc lựa chọn phải xuất phát từ mục tiêu ứng dụng và validation.

Ở confidence 0,5936, **macro precision 96,10% và macro recall 93,94%** từ số đếm theo lớp; micro tương ứng 95,99% và 93,76%. Native P/R có nội suy trên đường confidence nên có thể khác một chút số đếm trực tiếp tại một ngưỡng. Không trộn hai cách tính để kết luận tăng/giảm chất lượng.



Hình 3. Đường từng lớp và đường trung bình từ evaluator, sử dụng confidence sàn 0,001. Hình hỗ trợ quan sát đánh đổi precision–recall; không dùng đường test để lựa chọn ngưỡng triển khai.

7.4. Kết quả từng lớp



Bảng 7. Năm lớp có AP50–95 test thấp nhất; tên vẫn dùng ID cục bộ.

Lớp	Ảnh có GT	Hộp GT	P	R	AP50	AP50–95
class_20	25	47	92,72	87,23	92,53	66,80
class_33	49	74	89,79	91,89	91,53	70,94
class_29	37	41	96,66	90,24	92,14	72,84
class_47	43	89	85,87	81,93	84,23	74,53
class_49	53	66	92,20	89,57	93,68	75,68

Lớp `class_20` có AP50 cao hơn AP50–95 rõ rệt, cho thấy khi tăng yêu cầu chồng lấp hộp, kết quả giảm. Điều này gợi ý cần xem chất lượng định vị và quy tắc gán nhãn, không chỉ phân loại. `class_47` có cả P/R và AP50 tương đối thấp, cần rà soát nhận diện lẫn độ hạt của annotation. Mẫu lỗi lớn nhất của lớp này là một hộp GT bao cả khối trong khi dự đoán tách thành nhiều hộp nhỏ.

Bảng 8. Năm lớp có AP50–95 test cao nhất.

Lớp	Ảnh có GT	Hộp GT	P	R	AP50	AP50–95
class_61	71	78	98,29	98,72	99,47	96,73
class_16	94	111	98,31	99,10	99,50	95,95
class_64	66	74	98,65	98,67	99,49	95,44
class_8	54	54	97,53	98,15	99,21	94,75
class_7	61	96	99,20	100,00	99,50	94,60

Số mẫu GT đi cùng AP giúp tránh kết luận từ một tỷ lệ không có hỗ trợ dữ liệu. Không xếp hạng tên món khi chưa xác nhận mapping. Bảng đủ 68 lớp nằm ở Phụ lục A; CSV gốc chứa P/R, AP và TP/FP/FN.

7.5. Tốc độ suy luận

Lượt evaluator test batch 16 báo trung bình **0,065 ms tiền xử lý, 1,859 ms forward và 0,795 ms hậu xử lý mỗi ảnh**. Đây là thời gian nội bộ được chia theo batch; không bao gồm toàn bộ đọc file, tạo báo cáo và giao diện, nên không lấy nghịch đảo forward để quảng bá FPS ứng dụng.

Benchmark riêng dùng 100 ảnh test chọn đều theo danh sách file đã sắp xếp, chạy từng ảnh bằng 100 lời gọi predict riêng, FP16, `rect=False`, tensor 640×640. Warmup 3 ảnh cùng cấu hình không tính vào số đo.

Chỉ số	Số đo
Số ảnh / warmup	100 / 3
Forward trung bình / trung vị / p95 (ms)	7,234 / 6,640 / 9,727
Wall time trung bình (ms/ảnh)	10,890
Thông lượng wall time (ảnh/giây)	91,82

Chỉ số	Số đo
CUDA allocated cực đại (MiB)	80,15

Đây là một lượt đo local trên máy hiện có. Wall time có đọc ảnh và overhead Python/API, loại trừ tải model, warmup, vẽ kết quả, mạng và UI. Độ trễ có thể khác khi webcam, export TensorRT, chạy CPU hoặc xử lý đồng thời. Một lượt thử `rect=True` đổi shape đã cho độ trễ dao động lớn; số đo đó được giữ trong `benchmark_dynamic_shapes.json`, không thay cho cấu hình tensor cố định ở bảng trên.

8. PHÂN TÍCH LỖI VÀ KIỂM THỬ

8.1. Lỗi ở điểm vận hành đã khóa

Phân tích toàn bộ 3.284 ảnh test với confidence 0,5936 và IoU 0,50 cho thấy:

Quan sát	Số lượng
Ảnh có ít nhất một FP hoặc FN	382
Ảnh có GT và tất cả hộp ghép đúng, không FP/FN	2.829
Ảnh không có nhãn và không có dự đoán giữ lại	73
Tổng TP / FP / FN	5.316 / 222 / 354

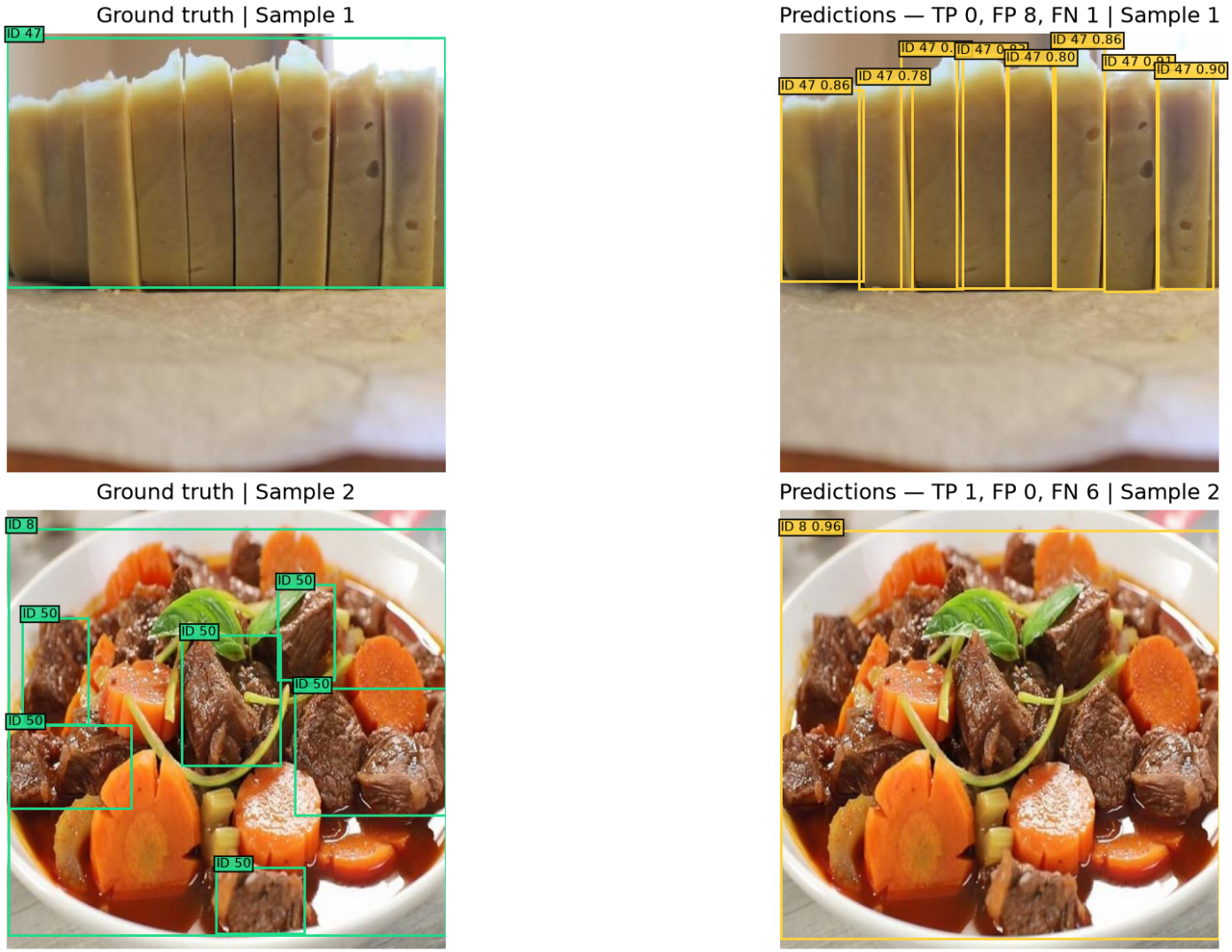
Theo annotation, có 75 ảnh nền test; 73 không có dự đoán giữ lại. Điều đó không chứng minh mô hình không phát hiện nhầm ở mọi ảnh ngoài 68 lớp. Nhãn rỗng hoặc thiếu annotation cũng có thể làm thay đổi cách ghi nhận lỗi.

Bảng 9. Phân nhóm hình học của 222 FP, không khẳng định nguyên nhân ngữ nghĩa.

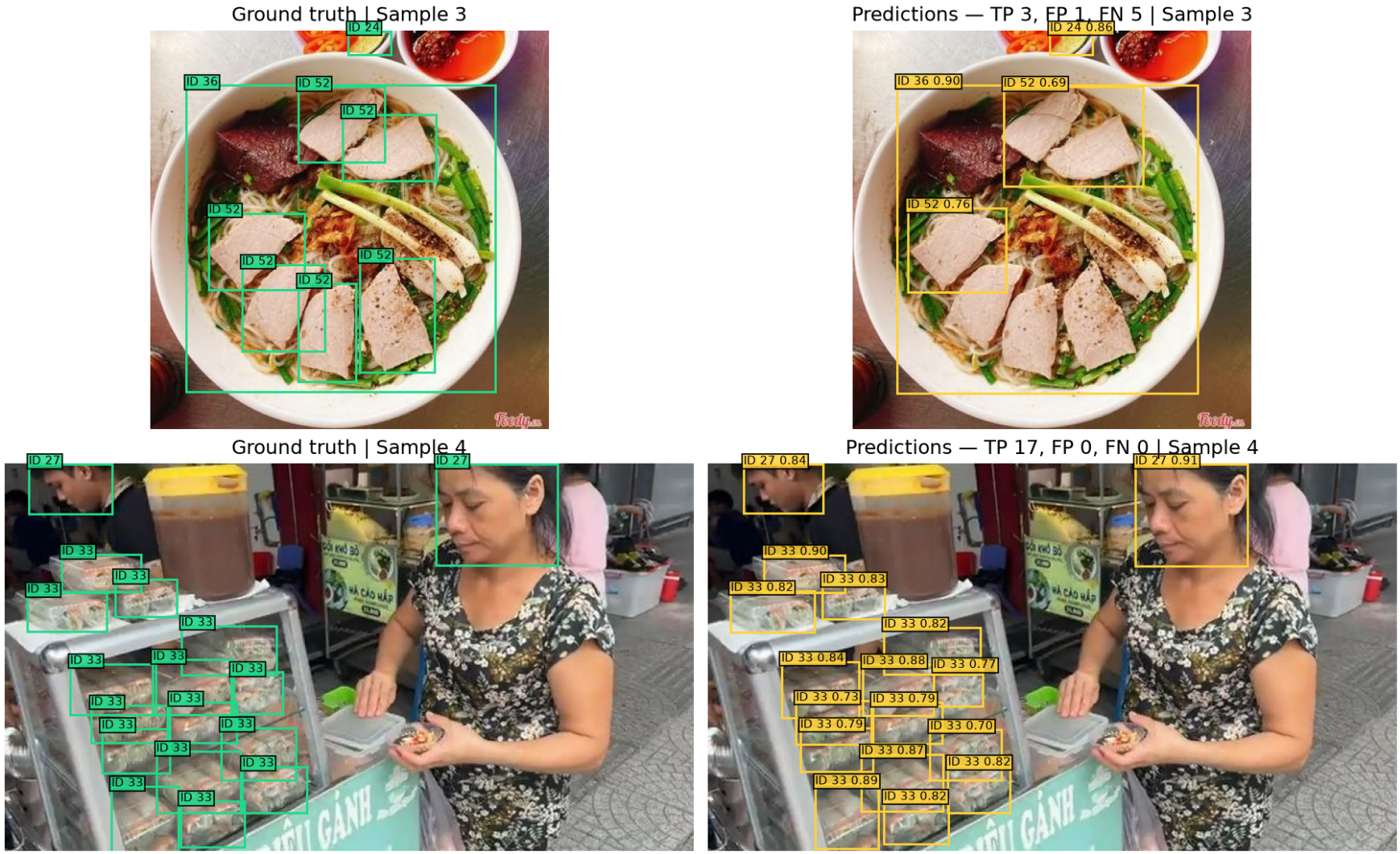
Điều kiện quan sát	FP
Không có GT nào đạt IoU ≥ 0,50	195
Có GT chồng lấp cùng lớp nhưng dự đoán không ghép được	16
GT có IoU cao nhất đạt 0,50 nhưng khác lớp	11

Nhóm 195 có thể gồm hộp lệch, khác độ hạt gán nhãn hoặc đối tượng chưa có annotation; không gọi tất cả là “nhận diện nhầm món”. Nhóm 16 liên quan đến nhiều dự đoán chồng nhau hoặc xung đột ghép cặp. Cần kiểm tra trực quan trước khi sửa nhãn hoặc thay mô hình.

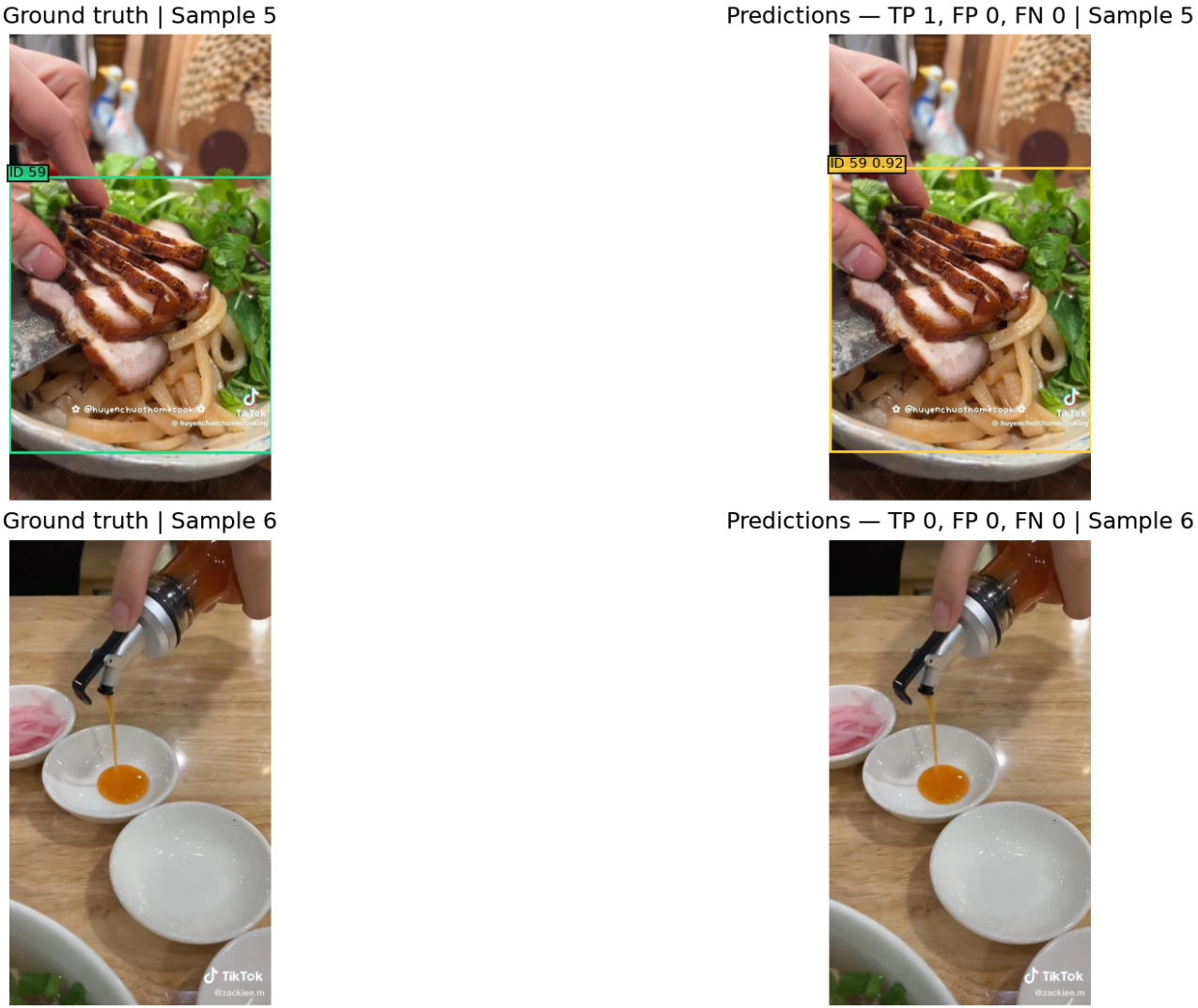
8.2. Minh họa và nhận xét cụ thể



Hình 5a. Mẫu 1–2: bên trái là GT, bên phải là dự đoán tại confidence 0,5936.



Hình 5b. Mẫu 3–4: bên trái là GT, bên phải là dự đoán tại confidence 0,5936.



Hình 5c. Mẫu 5–6: bên trái là GT, bên phải là dự đoán tại confidence 0,5936.

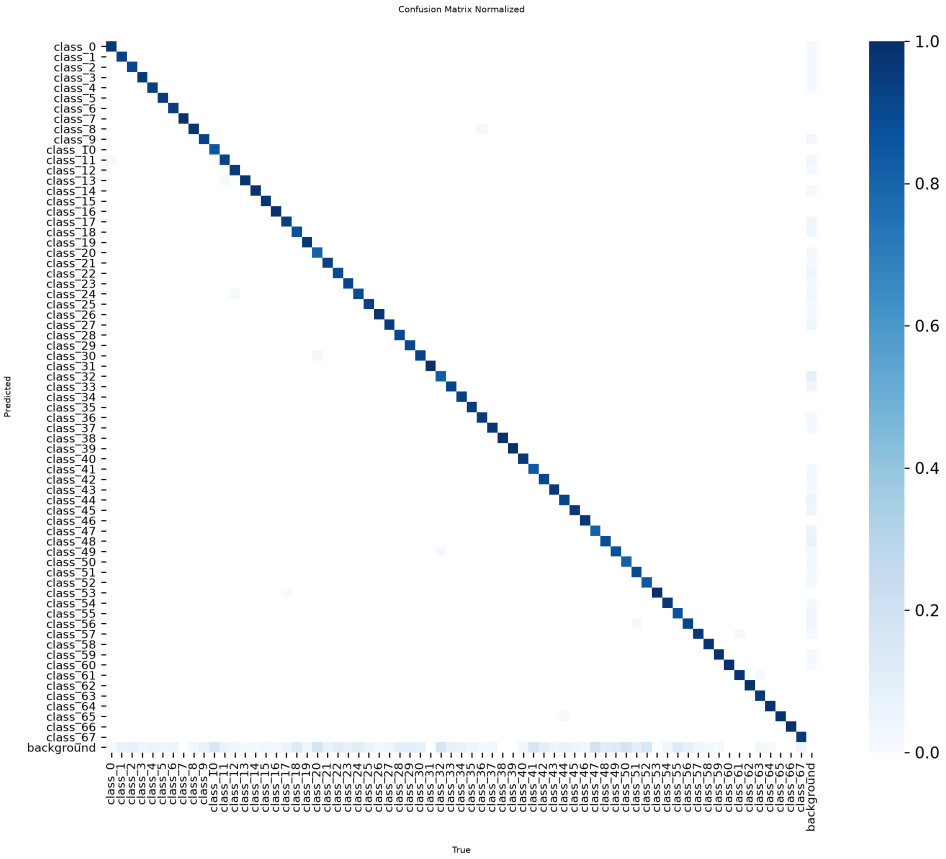
Hình 5. Mẫu 1–3 được chọn tự động theo số FP+FN cao nhất. Mẫu 4 là một ảnh nhiều GT được ghép đúng toàn bộ; mẫu 5 là một ảnh đúng khác trong danh sách; mẫu 6 là ảnh nhãn rỗng không có dự đoán. Đây là ví dụ phân tích có chủ đích, không phải mẫu ngẫu nhiên đại diện toàn test.

Mẫu	File	GT	TP	FP	FN
1	`10a510e484c0f1e6_jpg.rf.66d733789288361520f14cff185b6d2b.jpg`	1	0	8	1
2	`44_jpg.rf.ba355193b9a3368fe412332162610372.jpg`	7	1	0	6
3	`33_jpg.rf.57122550603c1d6c7fddd47b1113338b.jpg`	8	3	1	5
4	`yt-QzbNsSrMoKE-0038_jpg.rf.9a1ad84f41dc7a630ebb613aa7d25fb7.jpg`	17	17	0	0
5	`Download-8-_mp4-0040_jpg.rf.459bfe749ea347a81f4a58c134d98bce.jpg`	1	1	0	0
6	`Download-1-_mp4-0036_jpg.rf.926a3824b6d072a2e5e37e2dceda1d23.jpg`	0	0	0	0

Mẫu 1 có GT `class_47` bao cả khối, còn dự đoán tách thành tám phần; cả tám không đạt điều kiện IoU với hộp lớn. Cần thống nhất "một đối tượng" là cả khối hay từng miếng. Mẫu 2 giữ được hộp món lớn `class_8` nhưng bỏ các hộp thành phần `class_50`. Mẫu 3 cho thấy các hộp thành phần chồng lấp và một số hộp nhỏ bị bỏ sót.

Mẫu 4 đặc biệt đáng chú ý: GT `class_27` bao quanh khuôn mặt/người, cùng nhiều GT `class_33`. Quan sát này phù hợp với lớp người trong nguồn tham chiếu, nên **68 ID không được mô tả là 68 món ăn**. Việc nhiều hộp cùng một ảnh được ghép đúng cũng không thay thế đánh giá theo lớp hoặc tập ngoài miền.

8.3. Ma trận nhầm lẫn tại ngưỡng vận hành



Hình 6. Hàng dự đoán, cột GT. Ô nền bao gồm FP/FN theo quy tắc matrix. Bản đủ số đếm tại test_operating_confusion_matrix.csv.

Các cặp nhầm lớp hình học đứng đầu chỉ xuất hiện một lần mỗi cặp, ví dụ GT 36 → dự đoán 8, GT 20 → dự đoán 30, GT 32 → dự đoán 49. Vì hỗ trợ ít, chưa có cơ sở kết luận một cặp món cụ thể bị nhầm thường xuyên. Rà soát định vị, bỏ sót và annotation có ưu tiên cao hơn việc suy diễn tên món từ ID.

8.4. Kiểm thử và bằng chứng

Mã	Nội dung	Kết quả và bằng chứng
T01	Ghép ảnh–nhãn và kiểm tra định dạng	Đạt về cấu trúc; audit toàn bộ nhãn, 0 lỗi
T02	Đủ các ID được cấu hình	Đạt; 68/68 ID ở cả ba tập
T03	Trùng file byte giữa các tập	Không phát hiện; SHA-256 và lọc kích thước
T04	Đọc và đánh giá toàn validation/test	Đạt; 6.602 và 3.284 ảnh đã xử lý

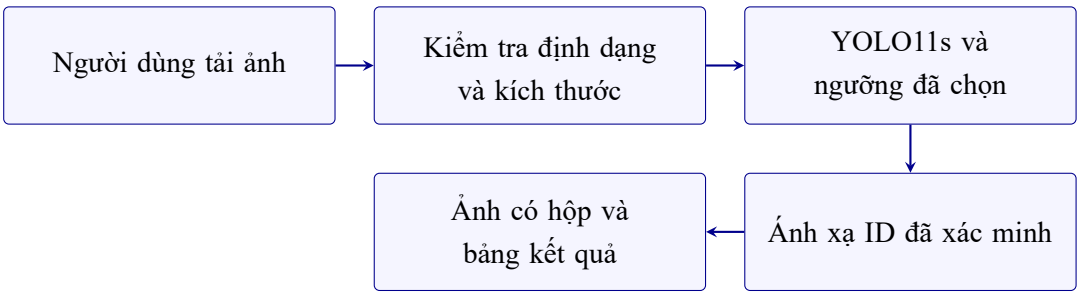
Mã	Nội dung	Kết quả và bằng chứng
T05	Checkpoint đúng kiến trúc và có thể suy luận	Đạt; cấu hình YOLO11s, 68 lớp và benchmark
T06	TP/FP/FN tại ngưỡng khóa	Đạt đối chiếu; đếm từ evaluator và ghép lại JSON trùng nhau
T07	Bộ nạp dữ liệu và resume batch	Đạt 3 kiểm thử; kiểm thử trong thư mục `tests`
T08	Tên món hiển thị đúng	Chưa kiểm chứng toàn bộ; thiếu mapping cục bộ gốc
T09	Giao diện Streamlit, upload, ảnh lỗi	Chưa thực hiện; chưa có app trong repo
T10	Dữ liệu ngoài miền, video/webcam và nhiều người dùng	Chưa thực hiện

"Đạt" ở T01–T07 có phạm vi cụ thể của phép kiểm tra, không đồng nghĩa hệ thống sẵn sàng triển khai sản phẩm. Bằng chứng máy đọc được nằm trong `reports/evaluation`, log chạy lưu ở thư mục `reports`.

9. THIẾT KẾ ỨNG DỤNG VÀ HƯỚNG PHÁT TRIỂN

9.1. Thiết kế giao diện đề xuất

Một ứng dụng Streamlit có thể nhận ảnh tải lên, giải mã ảnh, gọi model đã được cache và hiển thị hộp cùng bảng kết quả. `st.file_uploader` là thành phần phù hợp để nhận ảnh. Phần này là **thiết kế đề xuất**, không có ảnh giao diện hoặc trạng thái kiểm thử đạt vì repo hiện chưa triển khai app. 15



Thành phần	Yêu cầu thiết kế
Input	JPEG/PNG hợp lệ; giới hạn dung lượng, số pixel; thông báo khi không giải mã được
Model	Tải một lần; hiển thị phiên bản và cấu hình suy luận
Nhãn	Chỉ dùng mapping đã xác minh; tách lớp người khỏi bảng món ăn
Ngưỡng	Mặc định theo validation; cho phép thay đổi có giải thích đánh đổi P/R
Kết quả	Hộp, tên/ID, confidence; phân biệt không phát hiện với lỗi hệ thống

Thành phần	Yêu cầu thiết kế
Xuất dữ liệu	Ảnh kết quả và bảng CSV theo yêu cầu người dùng

Không tính calo trực tiếp từ diện tích hộp. Nhận diện tên món chưa cung cấp khối lượng, công thức chế biến hoặc khẩu phần. Nếu mở rộng dinh dưỡng, cần nguồn thành phần thực phẩm, đơn vị khẩu phần và phép kiểm chứng riêng; không sao chép các giá trị dinh dưỡng từ demo tham khảo thành số đo của hệ thống.

9.2. Thứ tự phát triển tiếp theo

1. Xác nhận mapping gốc, phiên bản dữ liệu và quy tắc gán nhãn món/nguyên liệu/người.
2. Kiểm tra ảnh gần trùng và nguồn video; chia lại theo nguồn trước mọi augmentation nếu cần.
3. Rà soát các lớp 20, 33, 29, 47, 49 và trường hợp gán nhãn khác độ hạt.
4. Thu thập một tập ngoài miền có ảnh điện thoại, nhà hàng mới, nền mới và vật thể ngoài danh sách lớp.
5. Huấn luyện baseline với cùng nguồn chia tập, cấu hình cố định và nhiều seed; so sánh YOLO11n/s/m bằng chất lượng, độ trễ và bộ nhớ thực đo.
6. Triển khai UI và kiểm thử file lỗi, không phát hiện, CPU fallback, video và tải đồng thời.
7. Chỉ thử TensorRT/ONNX sau khi khóa giao thức; đánh giá lại chất lượng sau export.

10. HẠN CHẾ, KHẢ NĂNG TÁI LẬP VÀ KẾT LUẬN

10.1. Hạn chế của kết quả

Hạn chế	Ảnh hưởng	Cách bổ sung
Thiếu metadata gốc và tên lớp cục bộ	Chưa xác minh tên của toàn bộ 68 ID	Lấy YAML/manifest của lần xuất dữ liệu
Chưa kiểm chứng chia theo nguồn	mAP có thể lạc quan nếu có ảnh gần trùng/video chung	Kiểm tra tương đồng và nhóm theo nguồn
Một run, có đổi batch và resume	Không tách được tác động từng cấu hình	Thực nghiệm đối chứng và nhiều seed
Ít GT ở một số lớp test	P/R/AP theo lớp có bất định lớn hơn	Thêm mẫu và bootstrap theo ảnh/nguồn
Không có tập ngoài miền	Chưa chứng minh khả năng dùng trên ảnh mới đa dạng	Đánh giá thực địa độc lập
Chỉ có nhãn hộp	Chưa có kiểm chứng khẩu phần hoặc dinh dưỡng	Tác vụ và dữ liệu đánh giá riêng
Chưa có app và khảo sát người dùng	Chưa đánh giá trải nghiệm sử dụng	Xây dựng UI rồi chạy kiểm thử có tiêu chí

Báo cáo không có khoảng tin cậy thống kê hoặc kết quả nhiều seed; không suy ra độ chắc chắn của

mô hình chỉ từ phần trăm cao. Với dữ liệu video, bootstrap theo khung hình riêng lẻ có thể đánh giá thấp bất định; nên nhóm theo video/nguồn.

10.2. Tái lập và quản lý hiện vật

Checkpoint đánh giá:

```
D:\vietfood\runs\detect\vietfood68_5070-2\weights\best.pt
SHA-256: b47a4666a1d73cf683df3bbc0750c1e92a58e3c7c925c5f3ceb769ea20f40741
```

Seed 42 được lưu nhưng `deterministic=False`, có AMP, benchmark cuDNN, multiprocessing và các lần resume. Vì vậy, chạy lại từ đầu không được bảo đảm trùng từng bit hay đúng từng số ở chữ số cuối. Cần lưu dataset manifest, phiên bản thư viện và cấu hình từng giai đoạn để tăng khả năng tái lập. Giữ cùng checkpoint và giao thức giúp tái lập lượt đánh giá rõ hơn việc cố tái tạo đúng toàn bộ đường training.

Số liệu báo cáo lấy từ CSV training, checkpoint, audit và evaluator thực chạy ngày 12/10/2026. Tài liệu trực tuyến dùng giải thích phương pháp và nghiên cứu liên quan, không thay thế số liệu thực nghiệm. Nội dung và số đo của FoodDetector được ghi là công trình tham khảo, không trình bày như sản phẩm của dự án này.

10.3. Kết luận

Đề tài đã hoàn thành quy trình huấn luyện và đánh giá YOLO11s 68 lớp trên RTX 5070, đạt **mAP50 97,23% và mAP50–95 85,32% trên test hiện có**. Precision native đạt 95,34%; tại confidence chọn từ validation, precision/recall micro là 95,99%/93,76%. Có bằng chứng từng lớp, phân tích FP/FN và trường hợp khác độ hạt gán nhãn để định hướng cải thiện.

Kết quả hỗ trợ sử dụng mô hình làm nền tảng cho ứng dụng nhận diện thực phẩm. Để khẳng định một hệ thống nhận diện món ăn hoàn chỉnh, cần bổ sung mapping lớp gốc, xác minh độc lập theo nguồn, đánh giá ngoài miền và triển khai giao diện. Báo cáo hiện phân biệt đầy đủ phần đã đo với thiết kế và công việc tiếp theo.

11. TÀI LIỆU THAM KHẢO

Các nguồn trực tuyến được tra cứu ngày **12/10/2026**. Với [3], phần đã truy cập gồm metadata và abstract công bố; không tuyên bố đã đọc toàn bộ chương trả phí.

1. J. Redmon, S. Divvala, R. Girshick, A. Farhadi, *You Only Look Once: Unified, Real-Time Object Detection*, 2016. arXiv:1506.02640.
2. T. T. Nguyen và cộng sự, *VinaFood21: A Novel Dataset for Evaluating Vietnamese Food Recognition*, 2021. arXiv:2108.02929.
3. V. H. N. Nguyen, B. T. Tran, M. V. Ton That, C. T. Vi, *Now I Know What I Am Eating: Real-Time Tracking and Nutritional Insights Using VietFood67 to Enhance User Experience*, SOICT 2024, CCIS 2352, tr. 456–470, xuất bản 2025. DOI:10.1007/978-981-96-4288-5_35.
4. G. Jocher, J. Qiu / Ultralytics, *Ultralytics YOLO11*, 2024. Tài liệu mô hình.
5. Ultralytics, *YOLO Architecture Explained*. Tài liệu kiến trúc.

6. Ultralytics, *YOLO Performance Metrics*. Hướng dẫn chỉ số.
7. T.-Y. Lin và cộng sự, *Microsoft COCO: Common Objects in Context*, 2014. Bài báo; giao thức detection evaluation.
8. Nam Nguyen, *VietFood67*, trang dữ liệu Kaggle; URL dùng slug `vietfood68`. Trang bộ dữ liệu.
9. nvhnam, *FoodDetector*, repo tác giả, nhánh v2. GitHub.
10. nvhnam, *class_names.py*, danh sách lớp tham chiếu của FoodDetector. Mã nguồn danh sách lớp.
11. Ultralytics, *Model Validation*. Tài liệu evaluator.
12. Ultralytics, *Model Training*. Tài liệu training.
13. PyTorch, *torch.utils.data*, phiên bản 2.8. Tài liệu DataLoader.
14. PyTorch, *Automatic Mixed Precision package – torch.amp*, phiên bản 2.8. Tài liệu AMP.
15. Streamlit, *st.file_uploader*. Tài liệu API.

PHỤ LỤC A. KẾT QUẢ TEST ĐỦ 68 LỚP

Đơn vị P/R/AP là %. P/R trong bảng này là native macro-F1 operating point của **test**, không phải số đếm tại ngưỡng khóa từ validation. Cột ảnh là số ảnh có GT của lớp; một ảnh có thể góp cho nhiều lớp. Tên `class_i` giữ nguyên trong `data.yaml` và checkpoint. Khi có mapping đã xác minh, có thể bổ sung tên mà không đổi ID hoặc tính lại AP.

Lớp	Ảnh có GT	Hộp GT	P	R	AP50	AP50–95
class_0	58	66	96,01	96,97	99,12	92,26
class_1	23	29	93,25	96,55	93,31	81,37
class_2	61	73	95,62	91,78	97,64	79,86
class_3	34	75	96,67	96,00	97,58	81,19
class_4	55	71	97,12	95,08	99,07	89,32
class_5	48	51	96,12	96,08	98,27	87,37
class_6	38	42	96,68	95,24	95,50	82,70
class_7	61	96	99,20	100,00	99,50	94,60
class_8	54	54	97,53	98,15	99,21	94,75
class_9	53	59	88,78	93,22	96,61	85,50
class_10	27	46	95,29	87,94	96,59	76,86
class_11	79	87	93,15	93,83	96,94	83,83
class_12	83	107	98,01	96,26	98,80	90,26
class_13	40	45	96,46	97,78	99,34	91,97
class_14	72	79	94,81	98,73	98,95	92,03
class_15	69	82	99,09	97,56	98,49	93,84
class_16	94	111	98,31	99,10	99,50	95,95
class_17	148	172	95,28	93,85	98,03	85,45

Lớp	Ảnh có GT	Hộp GT	P	R	AP50	AP50–95
class_18	64	131	91,86	88,55	93,83	80,61
class_19	30	57	98,24	97,92	98,62	86,92
class_20	25	47	92,72	87,23	92,53	66,80
class_21	56	57	94,70	94,74	96,88	87,85
class_22	92	137	94,59	89,41	94,28	80,63
class_23	64	86	94,13	93,18	97,46	86,73
class_24	94	110	91,34	89,09	95,49	80,02
class_25	54	64	95,32	95,48	98,97	90,46
class_26	52	56	95,80	98,21	98,50	86,68
class_27	244	323	97,18	95,98	99,20	86,91
class_28	27	32	96,67	90,72	98,96	83,89
class_29	37	41	96,66	90,24	92,14	72,84
class_30	47	54	94,40	93,63	98,21	84,20
class_31	27	31	96,52	100,00	99,50	92,11
class_32	80	120	84,06	83,52	91,57	77,58
class_33	49	74	89,79	91,89	91,53	70,94
class_34	61	76	98,63	94,56	96,62	80,41
class_35	55	67	97,28	95,52	97,71	82,08
class_36	42	46	90,84	95,65	97,00	81,43
class_37	20	41	93,15	97,56	96,43	82,81
class_38	55	60	97,86	100,00	99,48	84,33
class_39	70	82	99,51	100,00	99,50	93,28
class_40	49	54	99,81	96,30	99,09	91,56
class_41	42	54	91,91	85,19	95,54	79,75
class_42	73	88	94,39	90,91	96,93	84,84
class_43	36	49	94,97	95,92	97,97	83,21
class_44	68	174	93,82	92,53	97,49	83,48
class_45	55	96	91,21	95,83	98,00	85,06
class_46	59	65	100,00	98,28	99,48	91,62
class_47	43	89	85,87	81,93	84,23	74,53
class_48	213	234	93,34	89,89	95,71	78,48
class_49	53	66	92,20	89,57	93,68	75,68
class_50	48	64	92,92	82,01	93,21	78,41
class_51	66	81	94,82	90,47	95,65	82,60
class_52	67	84	94,85	87,74	97,28	77,92
class_53	55	59	96,33	100,00	99,48	91,81
class_54	44	55	93,67	96,36	96,28	89,06
class_55	68	83	90,22	88,95	97,00	80,91

Lớp	Ảnh có GT	Hộp GT	P	R	AP50	AP50–95
class_56	67	127	92,84	92,91	96,60	83,92
class_57	85	95	95,59	97,89	97,85	83,80
class_58	43	171	99,48	97,66	99,48	80,62
class_59	66	84	96,52	98,81	99,28	83,79
class_60	73	83	96,02	100,00	99,33	90,75
class_61	71	78	98,29	98,72	99,47	96,73
class_62	72	94	99,59	100,00	99,50	91,46
class_63	85	90	98,87	97,38	99,01	91,43
class_64	66	74	98,65	98,67	99,49	95,44
class_65	79	85	98,64	100,00	99,47	91,81
class_66	66	67	99,59	100,00	99,50	94,39
class_67	64	90	100,00	97,49	98,47	89,81

PHỤ LỤC B. LỆNH CHẠY VÀ TÁI ĐÁNH GIÁ

Chạy từ `C:\Users\lehoa\vietfood` bằng PowerShell, dùng môi trường đã cài phiên bản nêu ở Bảng 2.

Kiểm tra GPU và kiểm thử repo:

```
.\.venv\Scripts\python.exe check_gpu.py
.\.venv\Scripts\python.exe -m unittest discover -s tests -v
```

Huấn luyện một run mới với cấu hình cuối làm baseline:

```
.\.venv\Scripts\python.exe train.py --model yolo11s.pt --data data.yaml --epochs 150 --imgsz
↪  640 --batch 36 --workers 4 --prefetch 2 --cache off --project D:\vietfood\runs\detect
↪  --name vietfood68_5070_baseline_v2
```

Lệnh này bắt đầu run mới, không tái tạo chính xác lịch sử run đã đổi batch từ 28 sang 36. Không chạy lại training chỉ để xem kết quả hiện có.

Đánh giá đầy đủ và tạo hiện vật báo cáo:

```
.\.venv\Scripts\python.exe evaluate.py --model
↪  D:\vietfood\runs\detect\vietfood68_5070-2\weights\best.pt --data data.yaml --batch 16
↪  --project D:\vietfood\evaluation
.\.venv\Scripts\python.exe audit_dataset.py --dataset D:\archive\dataset
.\.venv\Scripts\python.exe analyze_results.py
.\.venv\Scripts\python.exe benchmark_inference.py
.\.venv\Scripts\python.exe render_report.py
```

`evaluate.py` tạo thư mục run mới khi tên đã tồn tại; đường dẫn thật được ghi trong `summary.json`. Các script phân tích hiện dùng dataset/run mặc định của dự án; cần sửa hoặc bổ sung tham số nếu chuyển máy. `render_report.py` dựng lại báo cáo từ mẫu, nên cập nhật mẫu nếu muốn giữ chỉnh sửa khi tái dựng.

Suy luận một ảnh với checkpoint đã huấn luyện:

```
.\.venv\Scripts\yolo.exe detect predict
↪  model="D:\vietfood\runs\detect\vietfood68_5070-2\weights\best.pt"
↪  source="D:\anh_mon_an.jpg" imgsz=640 rect=False conf=0.5936 iou=0.7 device=0 quantize=16
↪  project="D:\vietfood\predict" name="demo"
```

Thay `D:\anh_mon_an.jpg` bằng ảnh có thật. Log và hộp hiện dùng tên `class_i`; phải xác minh mapping trước khi hiển thị tên món cho người dùng.

PHỤ LỤC C. DANH MỤC HIỆN VẬT

Hiện vật	Đường dẫn / vai trò
Báo cáo ban đầu	Bản sao giữ nguyên
Mẫu báo cáo	report_template.md
Chỉ số và giao thức đánh giá	summary.json
Audit dữ liệu	dataset_audit.json
Kết quả từng lớp	Validation CSV, test CSV
Phân tích từng ảnh	test_image_errors.csv
Tổng hợp lỗi và mẫu minh họa	error_analysis.json
Benchmark cố định 640×640	benchmark.json
Benchmark đổi shape tham khảo	benchmark_dynamic_shapes.json
Phiên bản thư viện	Trước đánh giá, sau đánh giá
Log đánh giá / audit	evaluation-run.log, dataset-audit-run.log
Kết quả train đầy đủ	Bản chụp training_results.csv; bản gốc ở thư mục run trên D
Cấu hình cuối	Bản chụp training_args.yaml; bản gốc ở thư mục run trên D
Trọng số	`D:\vietfood\runs\detect\vietfood68_5070-2\weights\best.pt`
Dự đoán raw	`predictions.json` trong thư mục val/test tại `summary.json`

Biểu đồ nằm trong `reports/evaluation/figures`. Khi gửi Markdown, phải gửi kèm thư mục này và các bảng liên quan để hình và liên kết tương đối hoạt động. Dữ liệu ảnh lớn và checkpoint vẫn ở D; báo cáo không sao chép toàn bộ dataset lên C.

PHỤ LỤC D. THÔNG TIN CHƯA THỂ TỰ ĐIỀN

Nội dung thực nghiệm đã được điền từ số đo. Những thông tin chưa có bằng chứng được ghi rõ thay vì tạo dữ liệu giả:

- Họ tên giảng viên hướng dẫn và mẫu trình bày bắt buộc của học phần.
- Manifest nguồn, phiên bản tải/xuất, bước tạo ảnh train và quy tắc chia theo nguồn/video.
- Bảng ánh xạ 68 lớp gốc xác nhận cho đúng bản dữ liệu đang dùng.
- Kết quả UI, khảo sát người dùng, tập ngoài miền và độ chính xác dinh dưỡng, vì chưa thực hiện.